CS6004: Code Optimization for Object Oriented Languages

PA4: Beat the Interpreter

Report

S Ramachandran 23B1052

Abhilasha 23B1011

IIT Bombay

Spring 2026

Table of Contents

1 Problem Statement **2**

1.1 Part that we Tackled 2

1.2 Why is it useful? 2

2 Analysis Description **3**

2.1 Assumptions 3

2.2 Key Challenges 3

3 Comparing Performance of Transformed and Original Code **4**

4 Transformation Details **4**

1 Problem Statement

1.1 Part that we Tackled

Two main things were done in our implementation:

1. **Virtual call monomorphization.** A `virtualinvoke` or `interfaceinvoke` instruction is replaced with a `specialinvoke` (direct call) whenever static analysis can prove that exactly one concrete method implementation is reachable at that call site. Two independent sub-analyses provide evidence for this:
   - *Call-graph monomorphization (CG-mono):* if the whole-program call graph constructed by Soot contains exactly one outgoing edge from a call site, that edge is treated as the sole target.
   - *Points-to monomorphization (PT-mono):* if the points-to graph for the receiver local contains allocation sites of exactly one concrete class, the virtual dispatch is eliminated regardless of call-graph shape.
2. **Method inlining.** After a call is devirtualized to a `specialinvoke` or `staticinvoke`, the callee body is copied into the caller if the callee contains at most `INLINE_THRESHOLD` Jimple statements. The two passes are intertwined: each inlining round may expose new monomorphic sites, and vice versa, so the pipeline iterates until a fixed point (up to `MAX_PASSES = 10` iterations per method).

1.2 Why is it useful?

Monomorphization should cause a significant speed-up in the code in cases where a class with a function that has many child classes with said function overridden. This is because at runtime we are spared the cost of looking through the `vtable` of the class.

Java relies heavily on virtual dispatch. Every `virtualinvoke` at runtime requires a vtable lookup, an indirect branch, and potentially a cache miss. Under the JVM interpreter (no JIT), this cost is paid on every call. Monomorphization converts these to direct calls, and inlining then eliminates the call frame.

The combined optimisation is particularly valuable for:

- *Hot inner loops* over interface-typed collections (e.g. strategy, decorator, observer patterns) where the dynamic type is always the same despite the static type being abstract.
- *Short-lived object hierarchies* (factory results, builder returns) where the concrete type is fixed at the allocation site but hidden behind an abstract type in the program text

2 Analysis Description

The core analysis is a flow-sensitive, context-insensitive, allocation-site abstraction points-to analysis over Jimple. Each local variable is mapped to a set of abstract allocation sites identified by the unique NewExpr node in the jimple code (using an `IdentityHashMap` to distinguish identical `new` expressions at different program points).

For the interprocedural extensions, when `depth` > 0, the analysis descends into callees up to `MAX_INLINE_DEPTH` levels. Callee summaries are memoised in `methodSummaries` to avoid redundant work. A stub (empty graph) is inserted before recursing to break cycles in PTGraph. Return value sites are flowed into the caller's left-hand-side local using an assignment-compatibility check.

2.1 Assumptions

- **Closed-world assumption** The set of classes and methods is fixed at analysis time. Dynamic class loading, reflection, and `invokedynamic` are not modelled.
- **No aliasing through arrays** Array element stores (`x[i] = y`) are conservatively ignored; array loads union all heap targets of the base.
- Concurrent writes to shared fields are not modelled; the heap state is merged globally.
- **Constructors are not inlined** The JVM bytecode verifier requires that `putfield` instructions are only executed after `invokespecial <init>` completes on the receiver. Inlining a constructor body without also inlining the allocation violates this invariant, so constructors are excluded from the inline candidate set.

2.2 Key Challenges

Challenge 1 Dangling branch targets after inlining The original inliner cloned callee statements but did not update `UnitBox` references (used by `GotoStmt`, `IfStmt`, `SwitchStmt`) to point at the cloned units. This left dangling pointers into the callee's original unit chain, corrupting the CFG and causing `ArrayIndexOutOfBoundsException` in Soot's `PseudoTopologicalOrderer` during subsequent passes. The fix was to build a `unitMap` (original → clone mapping) and rewrite all `UnitBox` contents after cloning, followed by `callerBody.validate()`.

Challenge 2 Recursive inlining Without a guard, the inliner would attempt to inline a method into itself (directly or through a chain), diverging. A `currentlyInlining` set tracks active inline stack frames; any callee already in the set is skipped with a log message.

Challenge 3 Cycle breaking in interprocedural PT Mutually recursive methods (e.g. `isEven`/`isOdd`) would cause `buildPTGraphInter` to loop. A stub entry (empty `PTGraph`) is inserted into `methodSummaries` before recursing, so any re-entrant call returns the stub immediately and breaks the cycle.

3 Comparing Performance of Transformed and Original Code

Static metrics The number of `virtualinvoke`/`interfaceinvoke` call sites ("virtual call sites") and `specialinvoke`/`staticinvoke` call sites ("direct call sites") are counted in the Jimple IR before and after transformation. A reduction in virtual sites directly measures the success of monomorphization; a reduction in direct sites measures inlining (inlined calls are removed from the IR).

Wall-clock time. Transformed and original bytecode are emitted as `.class` files by Soot. Both are executed under interpreter mode, JIT disabled to ensure that the static optimization's effect is not masked by runtime devirtualization. Each test's `main` wraps the core logic in 50000 iterations. Five independent runs are timed using `System.nanoTime()`; the mean is reported.

Baseline fairness Both the original and transformed class files are produced by Soot (with and without the transformer), ensuring both went through the same Soot pipeline and any Soot-specific overhead is equal between the two measurements.

4 Transformation Details

```
    class Bottles {
    int x;
    Bottles() {x = 10;}
    Water ret_water() {return (new Water());}
    Bottles ret_bot() {return (new Wine());}
    void dummy() {System.out.println("bottle " + x);}
}
class Water extends Bottles {
    int y;
    Water() {y = 20;}
    Bottles ret_bot() {return (new Wine());}
    void dummy() {System.out.println("water " + y);}
}
class Wine extends Bottles {
    int z;
    Wine() {z = 30;}
    Bottles ret_bot() {return (new Bottles());}
    void dummy() {System.out.println("wine " + z);}
}
public class Test {
    static int runOnce() {
```

```
        Bottles b  = new Bottles();
        Bottles w  = b.ret_water();
        Bottles b2 = w.ret_bot();
        Bottles b3 = b2.ret_bot();
        b.dummy();
        w.dummy();
        b2.dummy();
        b3.dummy();
        return b.x + b2.x + b3.x;
    }
    public static void main(String[] args) {
        int iterations = 50000;
        long start = System.nanoTime();
        int acc = 0;
        for (int iter = 0; iter < iterations; iter++) {acc += runOnce();}
        long end = System.nanoTime();
        System.out.println("Result : " + acc);
        System.out.printf("Time   : %.3f ms%n", (end - start) / 1_000_000.0);
    }
}
```

Take the program given above as example. This demonstrates the monomorphization and inlining utility of the transformer. In the below image, on the right is the transformed code and we can see that the code is going down a function call for the `ret_bot()` and resolving to the correct class's `dummy()` function. We make the changes in the jimple file first and use Soot's utilities to get the class files for the benchmark and the transformed class. All these are cotained in the folder corresponding to the testcase for the

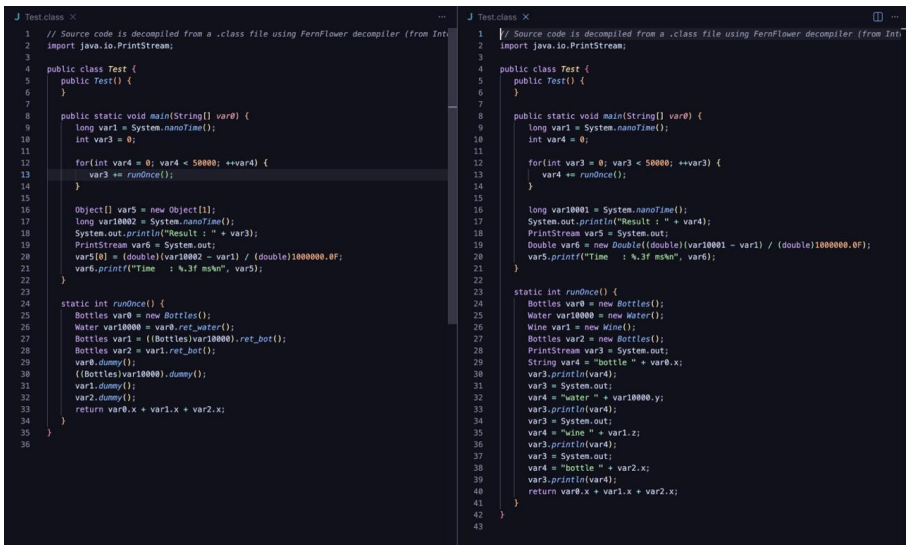


Figure 1: Example of transformed class